

How to Host the Debugging Challenge on HackerRank (Free)

A step-by-step guide for IEEE student-branch organizers. HackerRank Community contests are **free** for colleges, clubs and communities.

Menu names on HackerRank change from time to time. If a label below looks slightly different, look for the closest match. The overall flow stays the same.

Step 0 — Before you start (1 week before)

- Choose **one organizer account** (for example the IEEE student branch email). Every challenge you create is owned by this account.
- Get faculty email IDs ready so you can add them as **moderators**.
- Decide which question set to use: **Set A** (15 moderate + 10 hard) or **Set B** (18 easy + 4 moderate + 3 hard).
- Keep the answer-key PDF private. Only the organizer and moderators should have it.

Step 1 — Create a HackerRank account

1. Go to **www.hackerrank.com** and click **Sign Up** (use the developer / community sign-up, not "For Companies").
2. Verify the email address and log in.

Step 2 — Open the Administration panel

1. Open the profile menu (top-right avatar).
2. Click **Administration**. If it isn't in the menu, open **www.hackerrank.com/administration/contests** directly.
3. There are two tabs: **Manage Contests** and **Manage Challenges**.

Step 3 — Create the contest

1. **Manage Contests** → **Create Contest**.
2. Fill in:
 - **Contest Name:** IEEE Inter-Society Debugging Challenge 2026
 - **Start Time / End Time:** the event date, 2.5 hours long. Check the time zone (IST).
 - **Contest URL:** a short slug such as `ieee-debug-2026`
 - **Organization type:** College / University, and enter your college name
3. Click **Get Started / Create**.
4. On the **Details** tab, add a description, the prizes and the **rules**. You can paste the rules from Step 9.

Step 4 — Create each challenge (repeat for all 25)

1. **Manage Challenges → Create Challenge.**
2. **Details tab**
 - **Challenge Name:** e.g. "M1 – Single Neuron Activation"
 - **Description:** one line
 - **Problem Statement:** paste the statement and the **Concept** line from the questions PDF
 - **Input Format, Constraints, Output Format:** copy them from the PDF
 - **Tags:** e.g. debugging, python
 - Click **Save Changes**
3. **Test Cases tab**
 - Click **Add Test Case**. Paste the input, paste the expected output, and tick **Sample** for the sample case.
 - Add **6-10 hidden test cases** per problem, with edge cases like $n = 1$, zero, negative numbers and large values.
 - To make an expected output, run the **fixed program from the answer key** on your input and copy what it prints.
 - Optionally, you can upload a zip of `input/input00.txt ...` and `output/output00.txt ...` files.
 - Give every test case equal strength (points are split across the cases).
4. **Languages tab**
 - **Untick every language except the one the problem uses** (e.g. only Python 3).
 - In that language's **template / code stub** box, paste the **buggy code**. It then appears in the participant's editor when they open the problem. (If template head/body/tail sections appear, put everything in the **body**.)
 - Save.
5. **Settings tab**
 - **Max Score:** Set B uses 20 for easy, 50 for moderate and 100 for hard. Set A uses 50 for moderate and 100 for hard.
 - Keep the time limit at the default (e.g. 2 s for C++ and 10 s for Python/Java). HackerRank scales the limit per language.
 - Save.
6. Click **Preview / Try It**. Submit the **buggy** code and check that it **fails**. Then submit the **fixed** code and check that it **passes** every test.

Step 5 — Add the challenges to the contest

1. Go back to **Manage Contests → your contest → Challenges tab**.
2. Click **Add Challenge**, search by name, and add all 25.
3. Set each challenge's **Max Score** and **order**: Easy first, then Moderate, then Hard.

Step 6 — Add moderators

1. Open the **Moderators** tab in the contest, and in each challenge if needed.
2. Add faculty HackerRank usernames or emails. Moderators can view submissions and edit problems.

Step 7 — Advanced settings (fair play)

In **Advanced Settings** (options depend on account type), turn on whatever is available:

- **Leaderboard:** on. **Hide leaderboard in the last 15 minutes** adds suspense, if the option exists.
- **Tab-switch / copy-paste tracking** or **plagiarism check:** on, if offered.
- **Public vs. private:** keep the contest **private / invite-only**, or simply share the link only with registered students.
- **Scoring/Ranking:** total score, with time of the last accepted submission as the tie-breaker (this is the default).

Step 8 — Test run (1-2 days before)

1. Create a separate **dummy contest** with 2 problems.
2. Ask 3-4 volunteers from different departments to try it on the lab systems.
3. Check that the lab internet allows **hackerrank.com**, logins work, and the buggy code loads in the editor.

Step 9 — Rules to announce

1. Fix only the given code, in the given language. Fully rewritten solutions may be disqualified.
2. Internet search, AI tools and mobile phones are **not allowed**.
3. Score: hidden test cases give partial marks. Ties are broken by the earliest final accepted submission.
4. Teams of 2-3 from different departments (CSE, AI&DS, ECE, VLSI, Cyber), with one login per team.
5. The organizers' decision is final.

Step 10 — Contest day

- Open the lab 30 minutes early and make sure every team is logged in.
- Share the contest link: `www.hackerrank.com/contests/<your-slug>`
- Watch the **Submissions** and **Leaderboard** tabs live.
- Keep one volunteer for technical issues and one faculty member as moderator.

Step 11 — After the contest

1. Download the leaderboard (from the Leaderboard tab, or take a screenshot).
 2. Faculty spot-check the **top 10** submissions against the original code for the anti-rewrite rule.
 3. Announce the winners, overall and best per department group.
 4. Share the answer-key PDF with participants as a learning resource.
-

Alternatives (also free)

Platform	How	Notes
HackerEarth	Profile → <i>Host / Organize a challenge</i>	Similar setup. Confirm free availability for your account a week ahead
CodeChef	<i>Host Your Own Contest</i> (for institutions)	Needs approval a few days ahead
Codeforces Gym	Build the problems in Polygon , then run a <i>Mashup</i>	No editor stubs, so put the buggy code in the statement

Quick checklist

- ☐ All 25 challenges created, each with its buggy code template
- ☐ Only the correct language enabled per challenge
- ☐ 6–10 hidden tests per challenge, outputs taken from the fixed code
- ☐ Buggy code fails and fixed code passes (verified in Preview)
- ☐ Scores set, challenges ordered Easy → Hard
- ☐ Moderators added, rules posted, dry run done
- ☐ Answer key kept private